Loops continued….

for loop

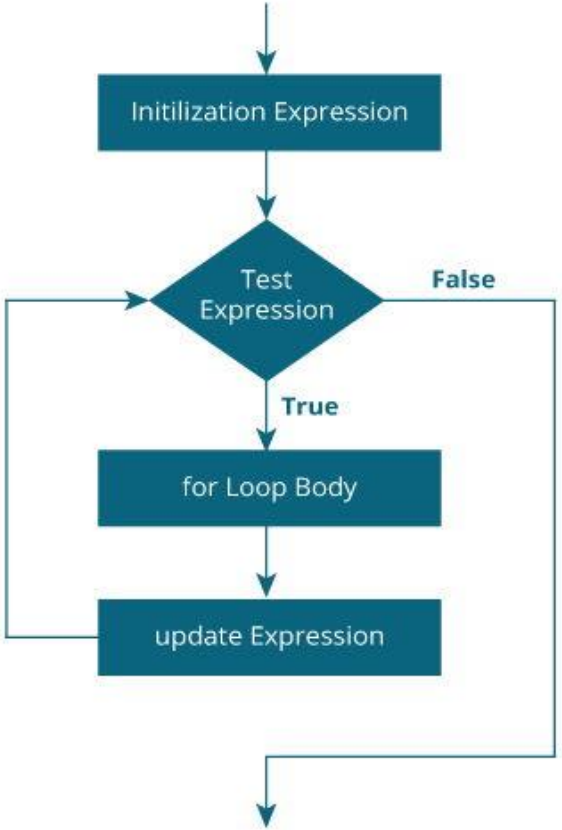
Syntax:

```
for (initializationStatement; testExpression; updateStatement)
{
    // statements inside the body of loop
}
```

How for loop works?

- The initialization statement is **executed only once**.
- Then, the test expression is evaluated. If the test expression is evaluated to false, the for loop is terminated.
- However, if the test expression is evaluated to true, statements inside the body of the for loop are executed, and the update expression is updated.
- Again the test expression is evaluated.
- This process goes on until the test expression is false. When the test expression is false, the loop terminates.

for loop – flow chart



for loop - example

```
// Print numbers from 1 to 10
#include <stdio.h>
int main()
{
   int i;
   for (i = 1; i < 11; ++i)
   {
        printf("%d ", i);
   }
   return 0;
}
```

for loop - example

```
//Program to calculate the sum of first n positive natural numbers
#include <stdio.h>
 int main()
{
   int num, count, sum = 0;
   printf("Enter a positive integer: ");
   scanf("%d", &num);
   // for loop terminates when count exceeds num
   for(count = 1; count <= num; ++count)
   {
        sum += count;
   }
   printf("Sum = %d", sum);
   return 0;
}
```

Infinite Loop

Infinite for Loop/NULL Parameter Loop

- This is also a kind of *for loop* where **the input parameters are not available** or do not exist by virtue of which the loop iterates/runs endlessly.

```
#include <stdio.h>
 int main()
{
      for (;;) // condition 1,2 and 3 are not entered
   {
      printf(" Infinite Loop");
   }
      return 0;
}
```

for Loop

Advantages of for Loop

- Provides code reusability
- Code size decreases
- Traversing in data structures like array and string becomes easy.

Disadvantages of for Loop

- Can’t skip any element while traversing
- Only a single condition is followed

The Do/While Loop

- A variant of the while loop.
- Execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.
- It is a form of an **exit-controlled or post-tested loop**
- The statements in the do…while loop will always be executed at least once no matter what the condition is.

Syntax:-

```
do {
  // code block to be executed
}
while (condition);
```

The Do/While Loop

Eg:-

```
void main()
{
    int i = 0;

    do
    {
        printf("%d\n", i);
        i++;
     }
while (i < 5);
}
```

The Do/While Loop

Eg:-

```
void main()
{
    int i = 10;

    do
    {
        printf("%d\n", i);
        i++;
     }
while (i < 5);
}
```

The Do/While Loop

Eg:-

```
void main()
{
    int i = 10;

    do
    {
        printf("%d\n", i);
        i++;
     }
while (i < 5);
}
```

Nested loop

- **Nesting of loops** is the feature in C that allows the looping of statements inside another loop.
- Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops.
- The nesting level can be defined at n times.
- You can define any type of loop inside another loop; for example, you can define '**while**' loop inside a '**for**' loop.

Syntax of Nested loop:

```
Outer_loop
{
   Inner_loop
  {
       // inner loop statements.
  }
     // outer loop statements.
}
```

Nested loop

- **Nested for loop**

```
for ( initialization; condition; increment )
        {
             for ( initialization; condition; increment )
              {
                   // statement of inside loop
              }
             // statement of outer loop
         }
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
#include <stdio.h>
int main()
{
    int limit = 5;
    int i = 1;
    while (i <= limit)
    {
        printf("\n");
        int j = 1;
        while (j <= i)
         {
            printf("%d ", j);
            j = j + 1;
          }
        i = i + 1;
    }
    return 0;
}
```

```
#include <stdio.h>
 int main()
{
    int i = 0;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 3; j++) {
            // This inner loop will break when
                                    i==3
            if (i == 3)
            {
                break;
            }
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Difference between break and continue

```
int i;
for (i = 0; i < 10; i++)
{
  if (i == 4)
  {
    break;
  }
  printf("%d\n", i);
}
```

Output:

0

1

2

3

```
int i;
for (i = 0; i < 10; i++) {
  if (i == 4) {
    continue;
  }
  printf("%d\n", i);
}
```

Output:

0

1

2

3

5

6

7

8

9